INTERVIEW PREP SERIES

MongoDB

Interview Questions & Easy Answers

---

52 questions · 19 marked ★ most asked

Covers NoSQL basics, documents and collections, CRUD, query and update operators, indexing, aggregation, schema design (embed vs reference), Mongoose, transactions, replication and sharding for MERN and backend interviews.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics

Q1. What is MongoDB? ★ Most asked

MongoDB is a NoSQL, document-oriented database. Instead of tables and rows, it stores data as flexible JSON-like **documents** (internally BSON) inside **collections**. It's the "M" in MERN and is popular because it's flexible, scales horizontally, and maps naturally to JavaScript objects.

Q2. What is the difference between SQL and NoSQL (MongoDB)? ★ Most asked

- SQL stores data in tables with fixed schemas; MongoDB stores documents with flexible schemas.
- SQL uses JOINs; MongoDB usually embeds related data or uses `$lookup`.
- SQL scales mostly vertically (bigger server); MongoDB scales horizontally (sharding).
- SQL is great for complex relations and strict transactions (banking); MongoDB is great for fast-changing, nested or large-scale data (catalogues, feeds, logs).

Q3. What are a document, a collection and a database in MongoDB? ★ Most asked

- **Document**: a single record, like `{ _id: ..., name: "Asha", age: 22 }` (like a row).
- **Collection**: a group of documents (like a table), e.g. `users`.
- **Database**: a group of collections.

Q4. What is BSON?

BSON (Binary JSON) is the binary format MongoDB uses to store documents. It is faster to scan than JSON and supports extra types like `Date`, `ObjectId`, `Decimal128` and binary data.

Q5. What is `_id` and ObjectId? ★ Most asked

Every document has a unique `_id` field, which is the primary key. If you don't provide one, MongoDB creates an **ObjectId**: a 12-byte value containing a timestamp, a random value and a counter, so it is unique and roughly sorted by creation time.

Q6. What are the advantages of MongoDB?

Flexible schema, fast development, natural JSON fit for JS apps, powerful queries and aggregation, built-in replication for high availability, sharding for scale, and a free cloud option (MongoDB Atlas).

Q7. When should you NOT use MongoDB?

When the data is highly relational with many joins and strict multi-table consistency rules, like accounting or banking systems. A relational database (MySQL/PostgreSQL) is usually a better fit there.

CRUD

Q8. What are the basic CRUD commands? ★ Most asked

```
db.users.insertOne({ name: "Asha", age: 22 })
db.users.insertMany([{ name: "Ravi" }, { name: "Neha" }])
db.users.find({ age: 22 })
db.users.findOne({ name: "Asha" })
db.users.updateOne({ name: "Asha" }, { $set: { age: 23 } })
db.users.updateMany({}, { $set: { active: true } })
db.users.deleteOne({ name: "Ravi" })
db.users.deleteMany({ active: false })
```

Q9. What is projection? ★ Most asked

Choosing which fields to return. 1 means include, 0 means exclude.

```
db.users.find({}, { name: 1, email: 1, _id: 0 })
```

Q10. How do you sort, limit and skip?

```
db.products.find().sort({ price: -1 }).skip(10).limit(10)
```

`1` is ascending, `-1` is descending. Skip + limit is used for pagination.

Q11. What is an upsert?

An update that inserts a new document if no match is found: `db.users.updateOne({ email: e }, { $set: { name: n } }, { upsert: true })`.

Q12. What is the difference between `find()` and `findOne()`?

`find()` returns a cursor to all matching documents (you can loop over it). `findOne()` returns only the first matching document, or `null`.

Q13. How do you count documents?

`db.users.countDocuments({ active: true })` for an exact filtered count.
`estimatedDocumentCount()` is faster but approximate and has no filter.

Operators

Q14. What are the common query operators? ★ Most asked

- Comparison: `$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`, `$in`, `$nin`
- Logical: `$and`, `$or`, `$not`, `$nor`
- Element: `$exists`, `$type`
- Others: `$regex`, `$elemMatch`, `$size`, `$all`

```
db.products.find({ price: { $gte: 100, $lte: 500 }, category: { $in: ["phone",
"tablet"] } })
db.users.find({ $or: [{ city: "Delhi" }, { age: { $lt: 25 } }] })
```

Q15. What are the common update operators? ★ Most asked

- `$set`: set a field. `$unset`: remove a field.
- `$inc`: increase/decrease a number (`{ $inc: { stock: -1 } }`).
- `$push`: add to an array. `$addToSet`: add only if not already there.
- `$pull`: remove matching items from an array. `$pop`: remove first/last.
- `$rename`: rename a field.

Q16. How do you query inside an array or a nested object?

Use dot notation for nested fields: `db.users.find({ "address.city": "Pune" })`. For arrays, `{ tags: "mongo" }` matches if the array contains "mongo"; `$elemMatch` matches when one array element meets multiple conditions.

Q17. How do you do a case-insensitive search?

`db.users.find({ name: { $regex: "asha", $options: "i" } })`. For big collections, a text index or Atlas Search is faster.

Indexing

Q18. What is an index and why is it important? ★ Most asked

An index is a special data structure (B-tree) that stores a field's values in sorted order with pointers to documents. Without an index, MongoDB scans every document (a collection scan). With an index, it jumps straight to the matching ones, like an index at the back of a book.

```
db.users.createIndex({ email: 1 }, { unique: true })
```

Trade-off: indexes take space and slow down writes slightly.

Q19. What are the types of indexes?

Single field, compound (multiple fields), multikey (array fields), text (search), geospatial (`2dsphere`), hashed (for sharding), unique, TTL (auto-delete after time), and partial indexes.

Q20. What is a compound index and the order rule?

An index on multiple fields, e.g. `{ category: 1, price: -1 }`. It helps queries that filter by `category`, or by `category` + `price`, but not by `price` alone (the prefix rule). Put equality fields first, then sort fields, then range fields (the ESR rule).

Q21. What is a TTL index?

A TTL (time-to-live) index automatically deletes documents after a set time, used for sessions, OTPs and logs: `db.otps.createIndex({ createdAt: 1 }, { expireAfterSeconds: 300 })`.

Q22. How do you check if a query uses an index? ★ Most asked

Use `explain`: `db.users.find({ email: "a@b.com" }).explain("executionStats")`. Look for `IXSCAN` (index used) vs `COLLSCAN` (full scan), and compare documents examined vs returned.

Aggregation

Q23. What is the aggregation pipeline? ★ Most asked

A way to process data in stages, where each stage transforms the documents and passes them to the next, like a factory line. Common stages: `$match` (filter), `$group` (group and calculate), `$project` (choose/reshape fields), `$sort`, `$limit`, `$skip`, `$lookup` (join), `$unwind` (flatten arrays), `$count`.

```
db.orders.aggregate([
  { $match: { status: "delivered" } },
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } },
  { $limit: 5 }
])
```

This gives the top 5 customers by total delivered order amount.

Q24. What is `$lookup`? ★ Most asked

`$lookup` performs a left outer join with another collection.

```
db.orders.aggregate([
  { $lookup: { from: "users", localField: "userId", foreignField: "_id", as:
"user" } }
])
```

Q25. What does `$unwind` do?

It splits an array field so each element becomes its own document. If a post has `tags: ["a","b"]`, `$unwind: "$tags"` creates two documents, one per tag. Often used before `$group`.

Q26. What are common group accumulators?

`$sum`, `$avg`, `$min`, `$max`, `$count`, `$push` (collect values into an array), `$addToSet` (collect unique values), `$first`, `$last`.

Q27. How would you find the second highest salary in MongoDB?

```
db.employees.aggregate([
  { $group: { _id: "$salary" } },
  { $sort: { _id: -1 } },
  { $skip: 1 },
  { $limit: 1 }
])
```

Grouping first removes duplicate salaries.

Schema design

Q28. Embedding vs referencing: when do you use which? ★ Most asked

- **Embed** (put data inside the document) when the data is read together, belongs to the parent, and is small/bounded. Example: a user's addresses, order items.
- **Reference** (store `_id` of another document) when data is large, unbounded, shared by many, or updated independently. Example: comments on a viral post, products in many orders.

Rule of thumb: "data that is accessed together should be stored together."

Q29. How do you model one-to-one, one-to-many and many-to-many relationships?

- One-to-one: embed (user and profile).
- One-to-few: embed an array (user and addresses).
- One-to-many (large): store the parent's `_id` in each child (post and comments).
- Many-to-many: arrays of references on one or both sides (students and courses).

Q30. What is the 16 MB document limit?

A single MongoDB document can't exceed 16 MB. That's why unbounded arrays (like all comments ever) should be stored in a separate collection. Large files go to GridFS or cloud storage.

Q31. What is schema validation in MongoDB?

Even though MongoDB is flexible, you can enforce rules at the database level using `$jsonSchema` validators on a collection (required fields, types, ranges). In MERN apps, Mongoose schemas usually handle this.

Mongoose

Q32. What is Mongoose? ★ Most asked

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js. It lets you define schemas, models, validation, default values, middleware (hooks) and relationships, making MongoDB code cleaner.

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  age: { type: Number, min: 18 },
}, { timestamps: true });

const User = mongoose.model("User", userSchema);
await User.create({ name: "Asha", email: "a@x.com" });
```

Q33. What is `populate()` in Mongoose? ★ Most asked

`populate()` replaces a referenced ObjectId with the actual document from another collection.

```
const postSchema = new Schema({ title: String, author: { type:
Schema.Types.ObjectId, ref: "User" } });
const posts = await Post.find().populate("author", "name email");
```

Q34. What are Mongoose middleware (hooks)?

Functions that run before or after certain operations. A common use is hashing a password before saving:

```
userSchema.pre("save", async function () {
  if (!this.isModified("password")) return;
  this.password = await bcrypt.hash(this.password, 10);
});
```

Q35. What are virtuals in Mongoose?

Fields computed on the fly and not stored in the database, e.g. `fullName` from `firstName` + `lastName`.

Q36. What does `timestamps: true` do?

It automatically adds and updates `createdAt` and `updatedAt` fields on every document.

Q37. What does `.lean()` do?

It returns plain JavaScript objects instead of full Mongoose documents, which makes read-only queries faster and lighter.

Q38. What is the difference between `findByIdAndUpdate` and `save()`?

`findByIdAndUpdate` updates directly in the database in one step (pass `{ new: true, runValidators: true }` to return the updated doc and validate). `save()` loads the document, changes it in JS, then saves it, which runs all `pre("save")` hooks and full validation.

Advanced

Q39. Does MongoDB support transactions? ★ Most asked

Yes. Single-document operations are always atomic. Since version 4.0, MongoDB supports multi-document ACID transactions on replica sets (and on sharded clusters since 4.2), using sessions with `startTransaction`, `commitTransaction` and `abortTransaction`. Good schema design (embedding) often avoids the need for them.

Q40. What is replication? ★ Most asked

Replication keeps copies of the same data on multiple servers called a **replica set**: one **primary** (handles writes) and several **secondaries** (copies). If the primary fails, the secondaries elect a new primary automatically. This gives high availability and data safety.

Q41. What is sharding? ★ Most asked

Sharding splits a large collection across multiple servers (shards) using a **shard key**, so each server holds part of the data. It provides horizontal scaling for huge data and high traffic. Choosing a good shard key (high cardinality, even distribution) is very important.

Q42. What is the difference between replication and sharding?

Replication = the same data copied on many servers (for availability). Sharding = different parts of the data on different servers (for scale). Production clusters often use both.

Q43. What are read and write concerns?

Write concern sets how many replica set members must confirm a write before it's considered successful (e.g. `w: "majority"`). Read concern sets how fresh/consistent the data you read must be. Higher levels are safer but slower.

Q44. What is the CAP theorem and where does MongoDB fit?

In a distributed system you can't fully guarantee Consistency, Availability and Partition tolerance at the same time; during a network split you must choose between C and A. MongoDB is usually described as CP by default, since it prefers consistency (writes go to the primary).

Q45. What is MongoDB Atlas?

MongoDB's fully managed cloud database service. It handles setup, backups, scaling and security, and has a free tier, which is perfect for student projects.

Q46. What are capped collections?

Fixed-size collections that overwrite the oldest documents when full, keeping insertion order. Useful for logs and caches.

Q47. What is GridFS?

A way to store files larger than 16 MB by splitting them into chunks across two collections. In most web apps, cloud storage like S3 is preferred.

Q48. What are change streams?

They let your app listen to real-time changes (insert, update, delete) in a collection, which is useful for live notifications and syncing data. Requires a replica set.

More frequently asked

Q49. What is the difference between `$push` and `$addToSet`?

`$push` always adds the value to the array, even if it's already there. `$addToSet` adds it only if it isn't already present, keeping the array unique (useful for likes or tags).

Q50. How would you design a schema for a blog with users, posts and comments? ★ Most asked

`users` collection with profile info. `posts` collection with `authorId` (reference to users), title, content, tags array and a `commentCount`. `comments` collection with `postId`, `userId`, text and `createdAt`, with an index on `postId`. Comments are kept separate because they can grow without limit; a few recent comments could also be embedded in the post for fast display.

Q51. What is the difference between `deleteOne` and `findOneAndDelete`?

`deleteOne` deletes the first matching document and returns only a count. `findOneAndDelete` deletes it and returns the deleted document, which is useful when you need to show or log what was removed.

Q52. What is the `$facet` stage?

It runs multiple aggregation pipelines on the same input in one query, e.g. getting both the paginated results and the total count for a search page.